

active-slam-rl

Reinforcement-learning-driven active SLAM
for confined underwater environments

A complete conceptual and visual walkthrough:
every file, every algorithm, and the physical reasoning behind each one

1. The big idea, physically

Regular SLAM is like exploring a dark cave with a flashlight while walking wherever your feet happen to take you — you build a map of whatever you shine light on, but you have no say in *where* you walk.

Active SLAM is choosing where to walk *because* it'll teach you something — deliberately swinging the flashlight back over a spot you're unsure about, or walking back to a junction you remember to check “wait, is this actually the same junction I saw an hour ago?” That deliberate choice — sense, decide, act, and let what you learn steer the next choice — is the whole loop this codebase implements.

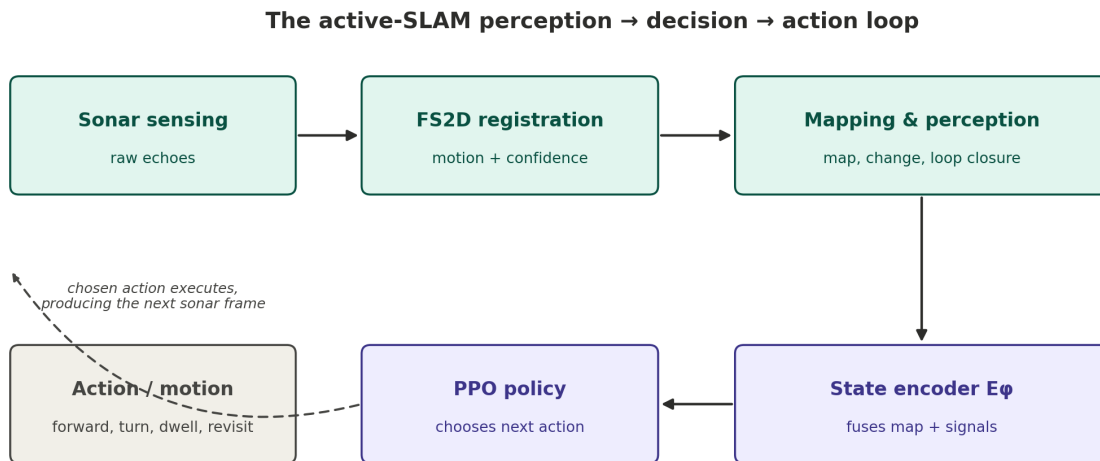


Figure 1. The full perception → decision → action loop. Every box below is a real, working module; the arrows are literal function calls in `env/sim_env.py`.

Think of the repository as five physical stations on an assembly line (sensing → registration → mapping/perception → thinking → acting), sitting inside a simulated factory floor, watched over by a quality-control department that grades every run.

2. Repository layout — what each folder physically is

```
active_slam_rl/
|-- Dockerfile, docker-compose.yml  a portable box with everything needed to run this
|-- configs/                        knobs: world size, sonar range, reward weights
|-- native/fs2d/                    scaffolding for the real C++ registration library
|-- src/active_slam_rl/
|   |-- registration/               "where did I just move to?"      (FS2D)
|   |-- mapping/                   "what does the world look like?"  (occupancy + change)
|   |-- perception/                "have I been here before?"      (loop closure)
|   |-- state/                     "what should the brain look at?"  (the encoder)
```

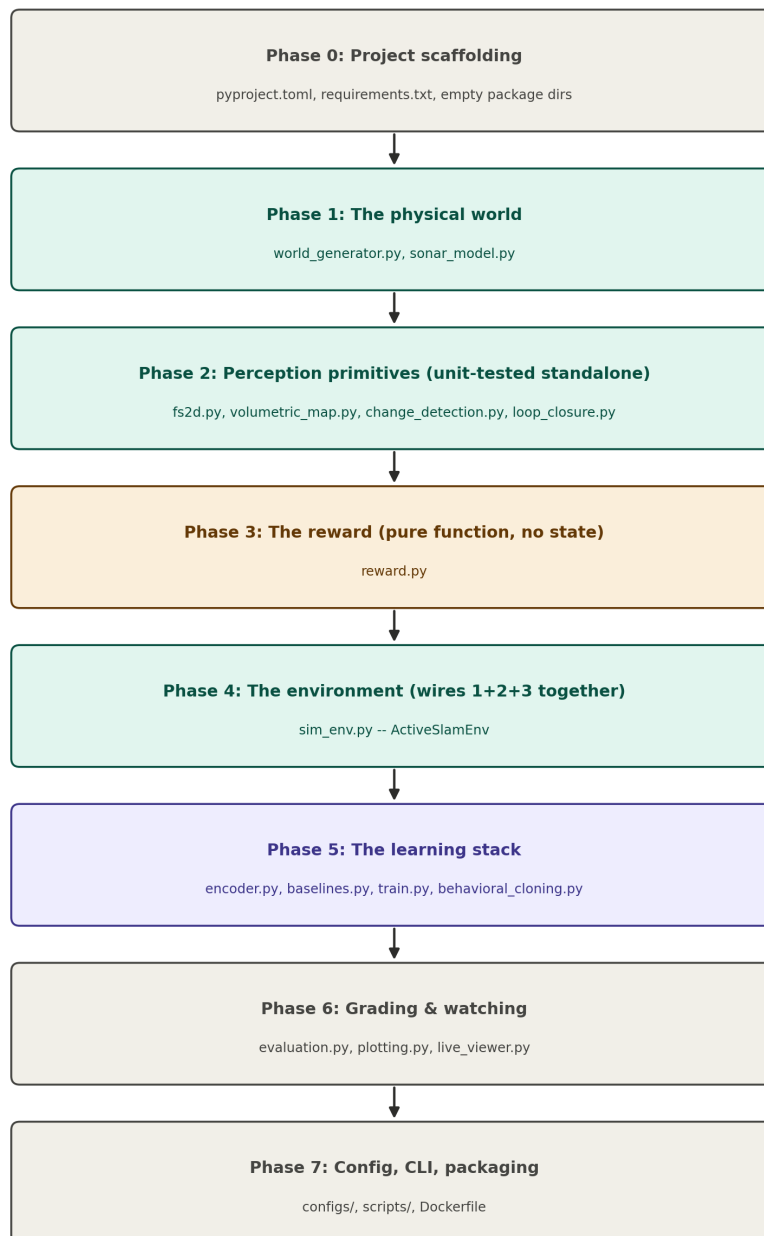
```
|      |-- env/           the physical world + the reward
|      |-- rl/           the decision-maker (PPO) + how it's trained
|      |-- metrics/      how we grade the whole thing
|      |-- visualization/ turning a run into something you can watch
|-- scripts/              the things you actually run
`-- tests/                proof each piece does what it claims
```

The next section walks through the order this was actually built in, with exact file and function references, so you can trace the dependency chain yourself. The sections after that walk through each station in the order data actually flows through one simulation step — read them the same way you'd trace a signal through a real physical system.

3. Build order: how this was actually put together

If you were writing this yourself, you wouldn't write it top-to-bottom in file order — you'd build whatever has *no dependencies* first, get it working and tested on its own, then build the next thing that only depends on what already exists. That's the order below: a working world and sensor before any perception, working perception before any environment, a working environment before any learning, and grading tools built last because you need something trained to grade.

Build order: each phase only depends on what came before it



Phase 0 — scaffolding

Nothing conceptual here: `pyproject.toml`, `requirements.txt`, and empty package directories under `src/active_slam_rl/` — this exists purely so `import active_slam_rl` works from anywhere once things are written.

Phase 1 — the physical world (nothing else can be tested without this)

Step	File : object	What it does / depends on
1.1	<code>env/world_generator.py</code> <code>WorldConfig</code> <code>TunnelWorld</code>	Procedurally carves a tunnel with a random walk. No dependencies on anything else in the project — pure NumPy. Built first because sensing and mapping both need a world to point at.
1.2	<code>env/sonar_model.py</code> <code>SonarConfig</code> <code>SonarModel</code>	Casts rays against <code>TunnelWorld.is_free()</code> , returns range readings and a rendered egocentric frame. Depends only on 1.1.

Phase 2 — perception primitives (each unit-tested standalone, before touching an environment)

Each of these four files can be developed and tested with synthetic data alone — you don't need a working simulator yet, just arrays that look like what the simulator will eventually hand them. That's exactly what `tests/test_registration.py` and `tests/test_mapping.py` do.

Step	File : object	What it does / depends on
2.1	<code>registration/fs2d.py</code> <code>FourierMellin-Registration</code>	Takes two frame-shaped arrays, returns a <code>RegistrationResult</code> . Doesn't need <code>TunnelWorld</code> or <code>SonarModel</code> to exist — tested against synthetic blob images first.
2.2	<code>mapping/volumetric_map.py</code> <code>OccupancyGrid</code>	Takes beam endpoints, updates a log-odds grid. Tested with hand-written beams before ever seeing a real sonar reading.
2.3	<code>mapping/change_detection.py</code> <code>compute_innovation</code> <code>compute_change_mask</code>	Pure functions over two <code>OccupancyGrid</code> snapshots (2.2). No new dependencies.
2.4	<code>perception/loop_closure.py</code> <code>SonarDescriptor</code> <code>LoopClosureDetector</code>	Depends on 2.1 (FS2D, for the final constraint) and needs frame-shaped arrays like 1.2 produces, but can be unit-tested with synthetic frames just like 2.1 was.

Phase 3 — the reward (written before the environment that calls it)

Step	File : object	What it does / depends on
3	<code>env/reward.py</code> <code>RewardWeights</code> <code>compute_reward</code>	A pure function: numbers in, a <code>RewardBreakdown</code> out. No environment state needed, so it's natural (and much easier to unit-test) to get right before it's wired into anything stateful.

Phase 4 — the environment: where phases 1–3 actually get wired together

Step	File : object	What it does / depends on
4	env/sim_env.py ActiveSlamEnv EnvConfig	The Gymnasium environment. Its step() method is just a sequence of calls into 1.1, 1.2, 2.1–2.4, and 3, in that order, plus bookkeeping for the true vs. estimated position. This file has nothing of its own to implement — it only orchestrates, so it could not have been written first.

Phase 5 — the learning stack (needs a working Gym env to attach to)

Step	File : object	What it does / depends on
5.1	state/encoder.py StateEncoder SB3StateEncoderExtractor	Its input shapes (patch channels/size, number of scalars) are read directly from ActiveSlamEnv's observation_space (4), so it couldn't be finalized before 4 existed.
5.2	rl/baselines.py RandomWalkPolicy FrontierBasedPolicy NextBestViewPolicy	The simplest possible thing that can act in the env (4) — built right after the env works, to sanity-check it end-to-end before any learning is involved at all.
5.3	rl/train.py rl/callbacks.py train(), MetricsLoggingCallback	Wires 5.1 into Stable-Baselines3's PPO as a custom features extractor, and logs episode metrics from env (4)'s info dict.
5.4	rl/behavioral_cloning.py collect_demonstrations behavioral_clone	Needs both 5.2 (an expert to imitate) and 5.3 (a trainable PPO policy to clone onto) — necessarily built last in this phase.

Phase 6 — grading and watching (needs something trained or heuristic to evaluate)

Step	File : object	What it does / depends on
6.1	metrics/evaluation.py rollout_episode EpisodeMetrics	Rolls out any policy from 5.2–5.4 against env (4) and computes completeness/ATE/RPE/etc. from its info dict.
6.2	metrics/plotting.py plot_training_curves plot_episode_dashboard	Turns 6.1's numbers and rl/callbacks.py's CSV into the PNGs shown throughout this document.
6.3	visualization/live_viewer.py record_rollout_frames render_gif	Same idea as 6.2, animated.

Phase 7 — configuration, entry points, and packaging

Step	File : object	What it does / depends on
7.1	configs/*.yaml utils/config.py load_config	Centralizes every hyperparameter used by phases 1–6 into one file, so nothing is hard-coded.
7.2	scripts/*.py	Thin CLI wrappers around train(), rollout_episode(), etc. — the only files meant to be run directly rather than imported.
7.3	Dockerfile docker-compose.yml tests/	Packaging and the regression tests that lock in each phase's behavior, including the reward-hacking fix's regression test in test_reward.py.

A true detail worth mentioning if you're presenting this: the reward-hacking exploit (a stationary vehicle farming the loop-closure bonus forever) was only discovered after Phase 5 was complete and a real training run had been evaluated — exactly the point in this build order where it would first become visible. The fix went back into Phase 3 (env/reward.py) and Phase 4 (env/sim_env.py), which is normal: later phases regularly reveal that an earlier one needs a revisit, and re-running the Phase-2-style unit tests afterward is what confirms nothing else broke.

Station 1 — Sonar sensing

File: env/sonar_model.py

Imagine standing in a foggy room and shouting, then listening for echoes. Each direction you shout in gives you one number back: “something bounced sound back at me from *this far* away.” Sonar is exactly this, done many times a second across a fan of directions (the imaging sonar’s $\sim 130^\circ$ cone) or a full slow sweep (the mechanical scanning sonar’s 360° rotation, like a lighthouse beam).

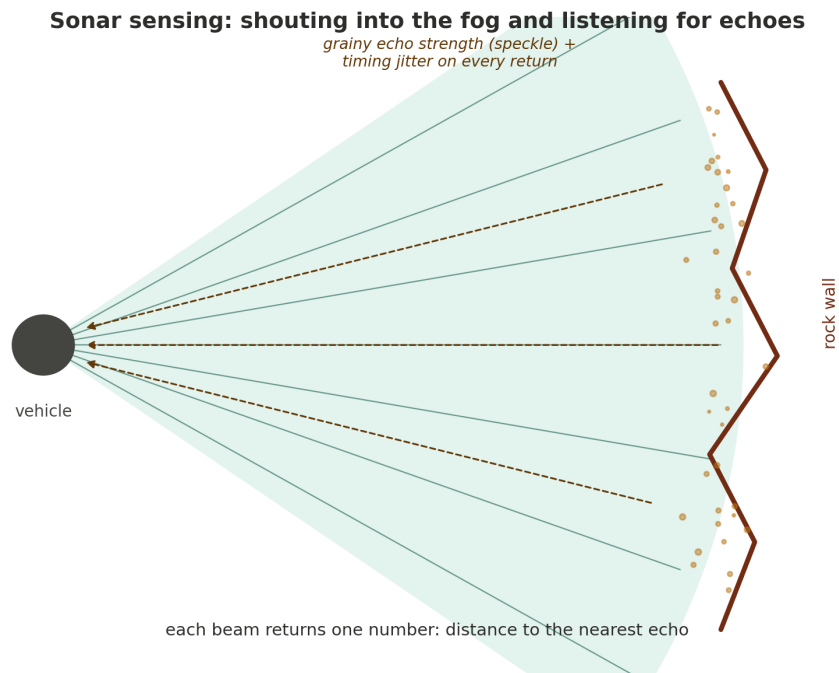


Figure 2. A fan of beams goes out, echoes come back — each carrying real acoustic noise.

Sound underwater doesn't bounce back cleanly — it partially reflects off rough surfaces, bounces around corners before coming back (multipath), and the echo strength is genuinely random even for a fixed surface (speckle, the same physics that makes a laser pointer show a grainy shimmer on a wall). The code models this honestly: **range_noise_std** adds real timing jitter, **multipath_ghost_prob** occasionally invents a fake echo from nowhere, and the rendered sonar frame has a grainy multiplicative intensity rather than a clean photo-like image. This matters later — it's *why* the registration step below can't just do ordinary photo-matching.

Station 2 — FS2D registration

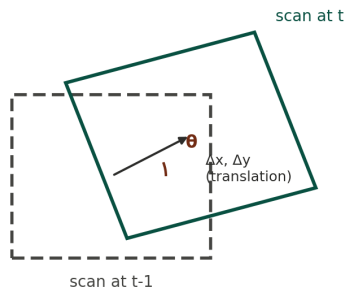
File: registration/fs2d.py

The physical question: given the echo pattern from a moment ago and the echo pattern now, how far did the vehicle actually move and turn?

Why not just overlay the two images and slide them around? Because of the speckle noise above — two echoes of the exact same wall look randomly different pixel-by-pixel, so directly comparing brightness values is like trying to match two patches of TV static. What doesn't randomly change is the coarse pattern of *where* the strong and weak echoes are — and that pattern survives much better if you look at it through a Fourier transform, which describes an image in terms of “how much repeating structure exists at this spacing and orientation” rather than “what's the brightness at this exact pixel.”

FS2D: finding motion between two noisy sonar frames

Two misaligned sonar frames



Log-polar re-plot: rotation = shift

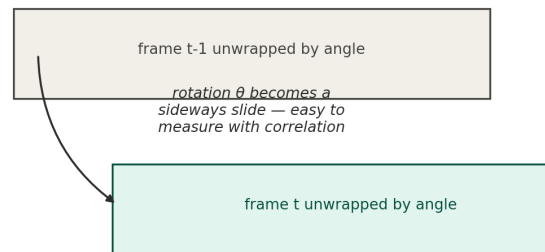


Figure 3. Left: two misaligned scans, related by a rotation θ and a translation $(\Delta x, \Delta y)$. Right: re-plotting in log-polar coordinates turns the rotation into a simple sideways shift.

The rotation trick. If you rotate a picture, its Fourier fingerprint rotates by exactly the same amount. Re-plot that fingerprint on a log-polar grid (angle, and log of distance from center) and a *rotation* becomes nothing more than *sliding sideways* along the angle axis — this is the “Mellin” half of Fourier-Mellin. Sliding is something you can measure fast and robustly with correlation.

Translation, after de-rotating. Once one frame is un-rotated to match the other's orientation, finding how far it moved sideways becomes a “slide the picture until it lines up best” problem, solved with **phase correlation** — the frequency-domain version of the same idea, naturally robust to speckle because it uses *phase* (where the pattern's peaks and troughs sit) rather than noisy amplitude.

Confidence and covariance — an honesty check. After finding the best alignment, ask: was there one clear, sharp best answer, or did many alignments look almost equally plausible? A sharp peak means real agreement — high confidence. A flat, ambiguous peak (two scans of featureless open water, where everything matches everything) means the registration is guessing, so the reported uncertainty should be

large. This confidence score (q_t) is exactly what flows into the robot's “brain” later, so it can learn “when my last registration was shaky, be more careful.”

Registration confidence = how sharp the correlation peak is

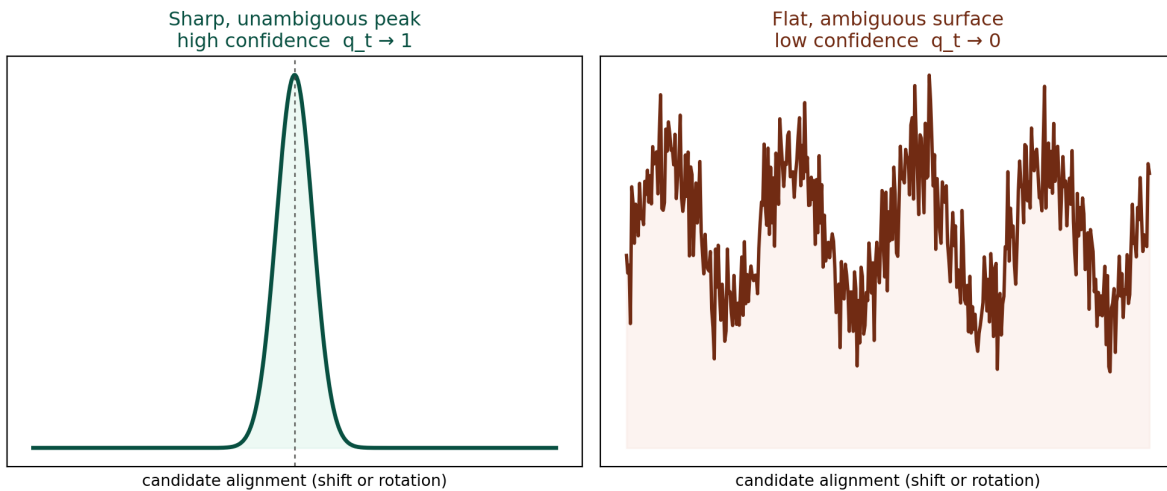


Figure 3b. Confidence isn't guessed — it's read directly off the shape of the correlation surface FS2D already computed to find the alignment in the first place.

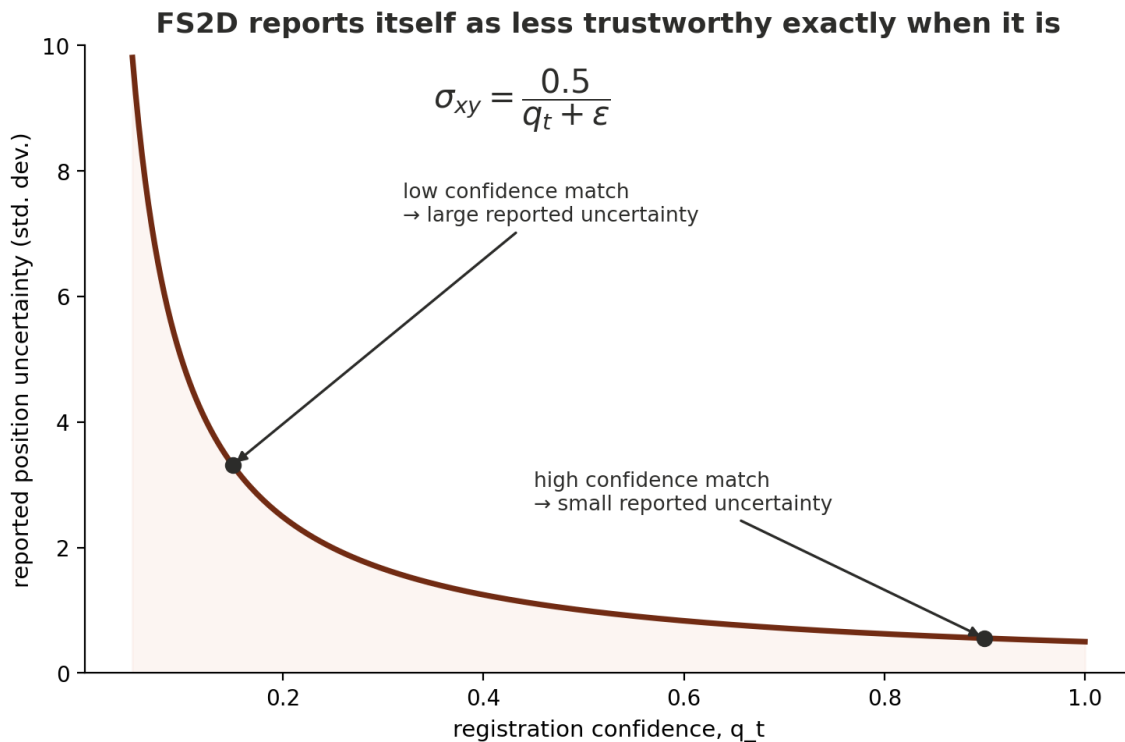


Figure 3c. The uncertainty handed downstream isn't a fixed number — it grows exactly when confidence drops, so a shaky registration is honestly reported as shaky.

Why two implementations exist: the real Constructor University FS2D library is compiled C/C++, and it can't be reimplemented without its source. FourierMellinRegistration is a genuine, working

version of the algorithm itself (not a stub), so training works today; NativeFS2DBinding is a documented, empty doorway (native/fs2d/) for the real library to be dropped in later without touching anything downstream.

Station 3 — Mapping & perception

3a. The volumetric map — building belief, not certainty

File: `mapping/volumetric_map.py`

Instead of a map that flatly says “wall” or “not wall,” imagine a huge grid of tiny voting booths, one per patch of the world, each holding a running tally of “occupied” votes vs. “free” votes cast by every sonar beam that ever passed through or bounced off that spot. A beam that passes cleanly through a cell casts a “free” vote; a beam that echoes off a cell casts an “occupied” vote. Cells nobody has looked at yet stay at “no idea, 50/50.”

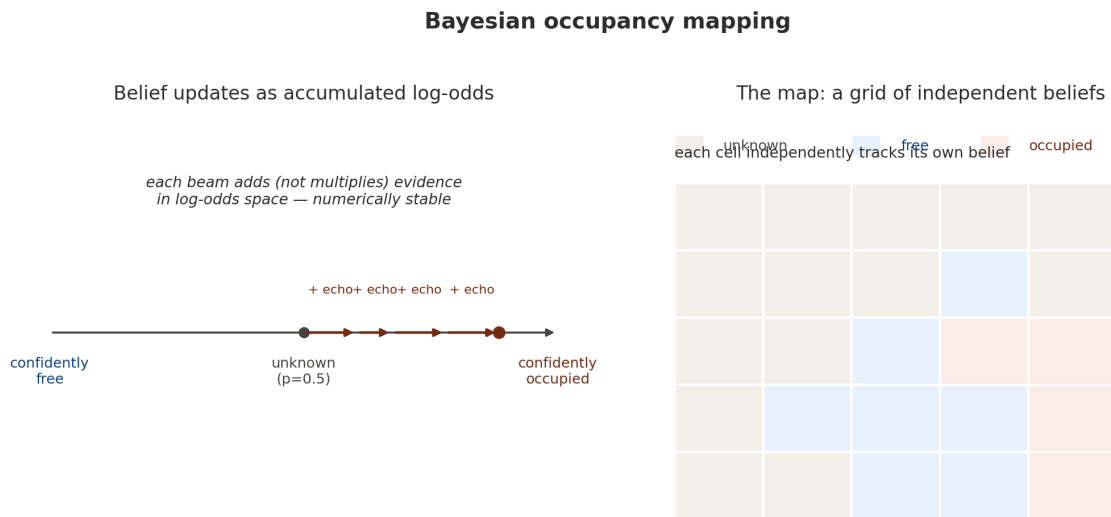


Figure 4. Left: evidence accumulates additively in log-odds space. Right: the map is a grid of independently-updated beliefs.

Why log-odds instead of raw probability: multiplying many small probabilities together underflows toward zero almost immediately. Instead, the code adds up evidence in log-odds space (0 = totally unsure, positive = leaning occupied, negative = leaning free), only converting back to a 0–1 probability when something needs to read it — the same trick as using decibels for sound instead of raw pressure ratios: addition instead of multiplication, so nothing blows up numerically.

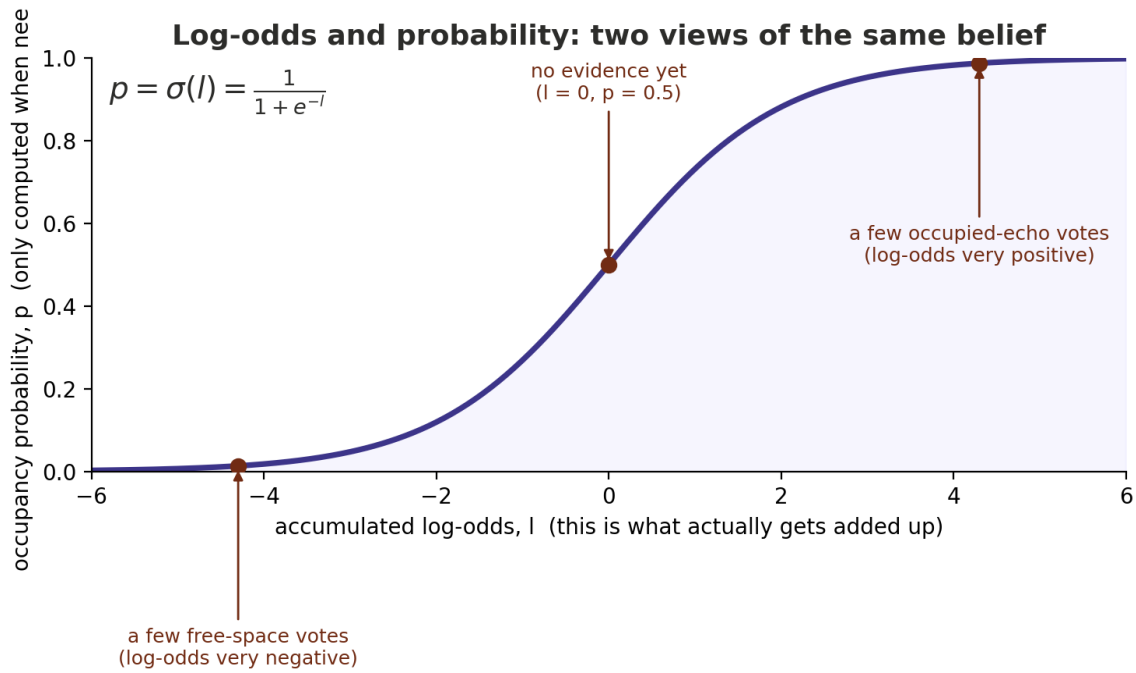


Figure 4b. The same belief, two representations: log-odds (unbounded, safe to add) and probability (bounded 0-1, easy to interpret). The sigmoid converts between them.

Entropy — turning “uncertain” into a number you can reward. A cell at 50/50 has maximum entropy (genuine ignorance); a cell resolved to confidently-occupied or confidently-free has near-zero entropy. Summing this over the whole grid gives one number: how much we still don't know. This is what the reward actually cares about reducing — not “did the vehicle move somewhere new,” but “did an action genuinely teach us something,” whether what it taught was “there's a wall here” or “this is open water.”

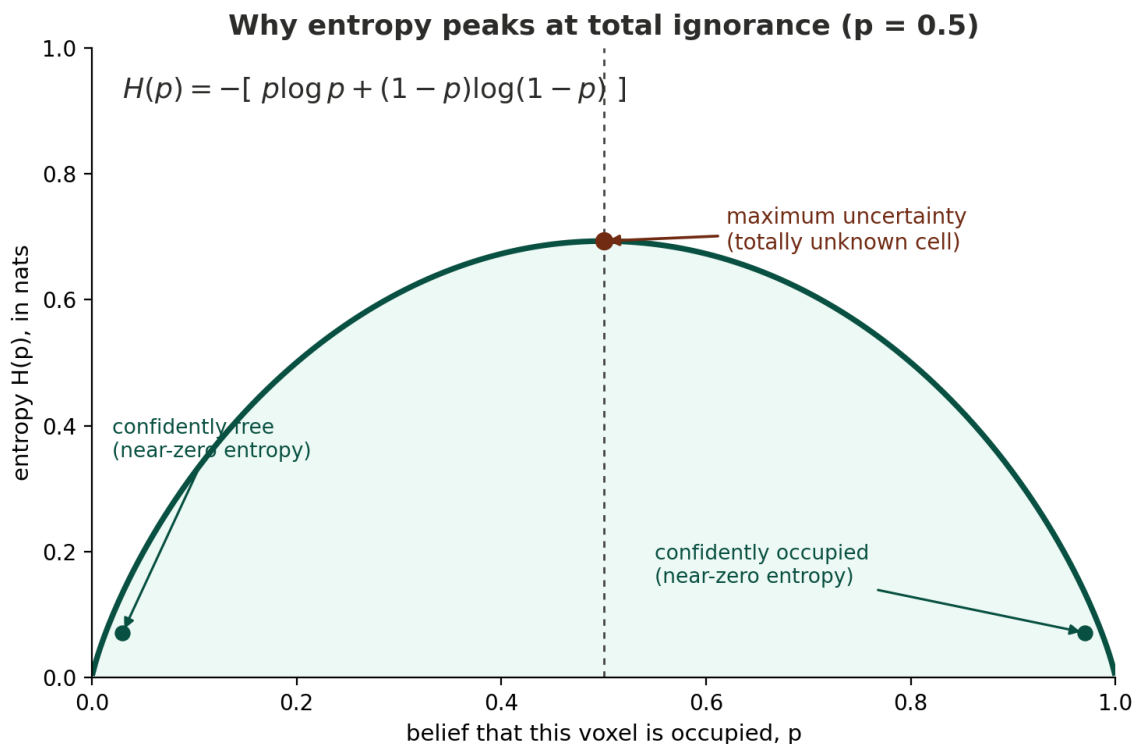


Figure 4c. The entropy function itself, plotted — zero at total certainty, maximum exactly at total ignorance. This curve, summed over the whole grid, is $H(M_t)$.

3b. Change detection — telling real change from noise

File: mapping/change_detection.py

Imagine checking a thermometer twice and seeing it jump 2 degrees. Is that a real temperature change, or just how jittery this thermometer normally is? You can't tell from the jump alone — you need to know the thermometer's *typical* jitter to judge whether 2 degrees is noise or a genuine event.

Change detection: is this a real surprise, or just noise?

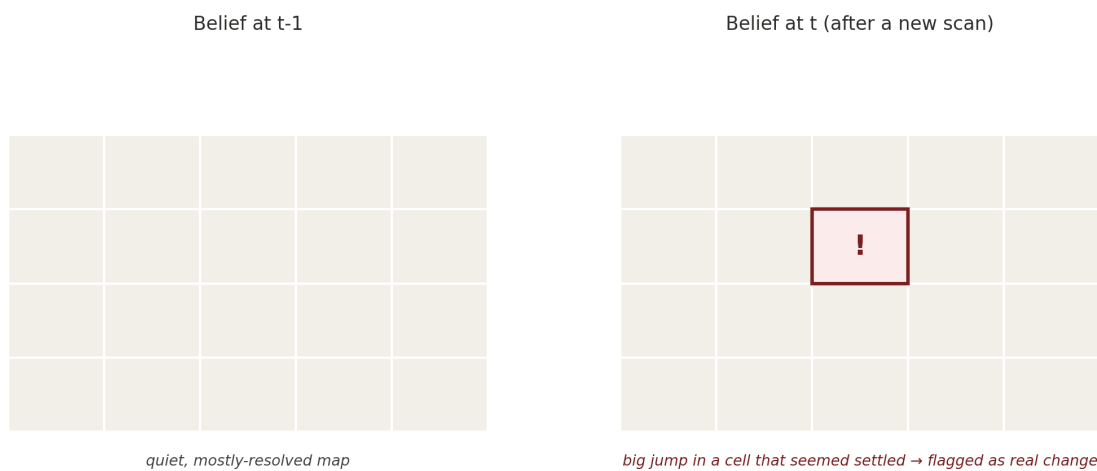


Figure 5. The same size belief-jump is unremarkable in an already-uncertain cell, but a real surprise in a cell that seemed settled.

Change detection: is this jump bigger than expected noise?

denominator: how much jumping
around was already expected here

$$\eta_t^v = \frac{|p_t(v) - p_{t-1}(v)|}{\sqrt{\text{Var}[p_t(v)] + \varepsilon}}$$

numerator: how much
the belief just jumped

big →

flag as real change

Figure 5b. The actual formula behind Figure 5: divide the size of the jump by how much jumping was already expected, given how uncertain that cell was.

The code does exactly this per grid cell: it takes the size of the belief's jump between one step and the next, and divides by how uncertain that cell already was. A big jump in a cell that was already uncertain is expected noise; the *same size* jump in a cell that seemed settled is flagged as a genuine surprise. Flagged cells become explicit “go double-check this” targets, both for the reward (go resolve it) and for the robot's state (so it can literally see where the map is currently shaky).

3c. Loop closure — recognizing “I’ve been here”

File: `perception/loop_closure.py`

Every registration step has some small error, and small errors compound over a long path — exactly like a person blindfolded and told to walk in a straight line. There's no way to know *how much* they've drifted from odometry alone, because odometry is the thing that's drifting. Without GPS (which doesn't work underwater), the only fix is recognizing a place you've been before and using that recognition to correct the accumulated error.

Loop closure: recognizing a place corrects accumulated drift

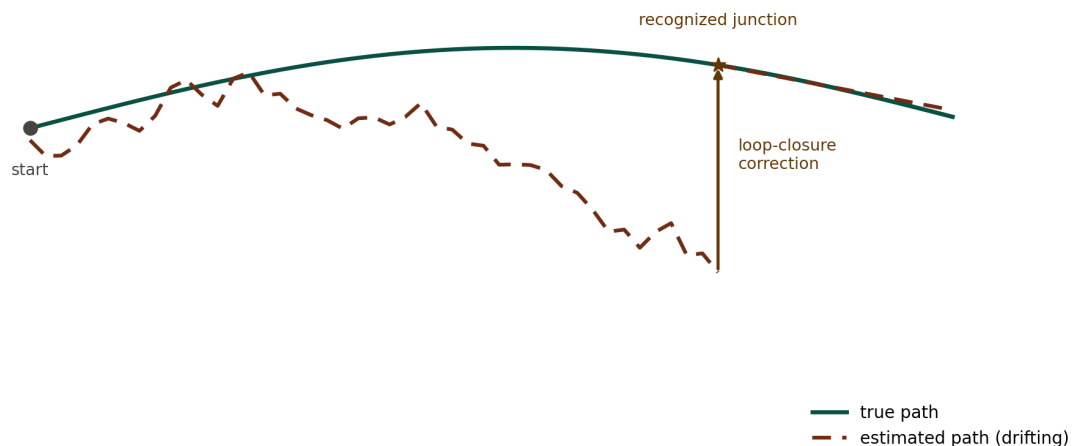


Figure 6. The estimated path (dashed) drifts away from the true path (solid) — until a recognized junction lets the estimate snap back.

Each visited spot gets a compact fingerprint of its sonar frame — a histogram of echo strength by *distance from the sensor*, deliberately built to ignore which direction the vehicle was facing, so a junction can be recognized even walking through it backwards relative to the first visit. Comparing the current fingerprint against every stored one and taking the best match gives a similarity score fed straight into the robot's state, so it can learn to act on that feeling of familiarity, not just a hard yes/no.

Once a match clears a confidence threshold, FS2D re-registers the current scan against the stored one from that earlier visit, producing a real geometric constraint: “relative to that earlier spot, I should be *here*, not where my drifted estimate says.” The position estimate is then pulled toward that corrected answer, weighted by how confident the match was. A full production SLAM system would spread this correction back across the entire remembered path (a “pose graph” optimization); this codebase does the honest, documented simplification of correcting just the current position — enough to see drift visibly shrink, not a substitute for a real backend optimizer before real deployment.

Station 4 — The state encoder: building the robot's “gut feeling”

File: state/encoder.py

The policy can't be shown the raw sonar image or the entire map — that's too much, too variable in size, and mostly irrelevant at any given instant. It needs a small, fixed-size summary: not “here is everything,” but “here is what matters right now.”

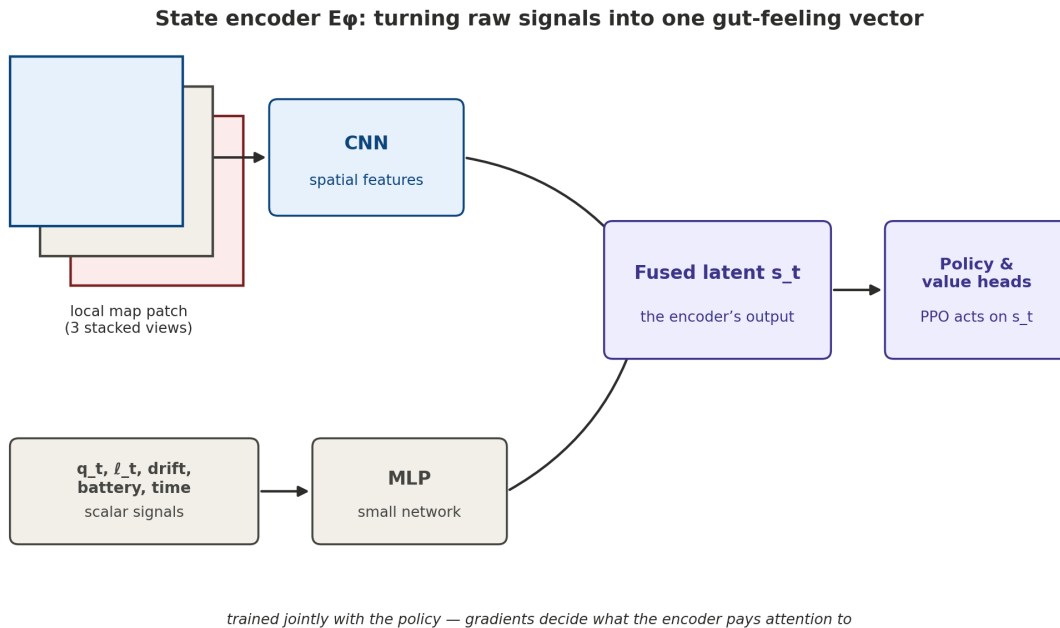


Figure 7. A small convolutional network reads a local map crop; a small MLP reads a handful of gut-feeling numbers; both get fused into one latent vector.

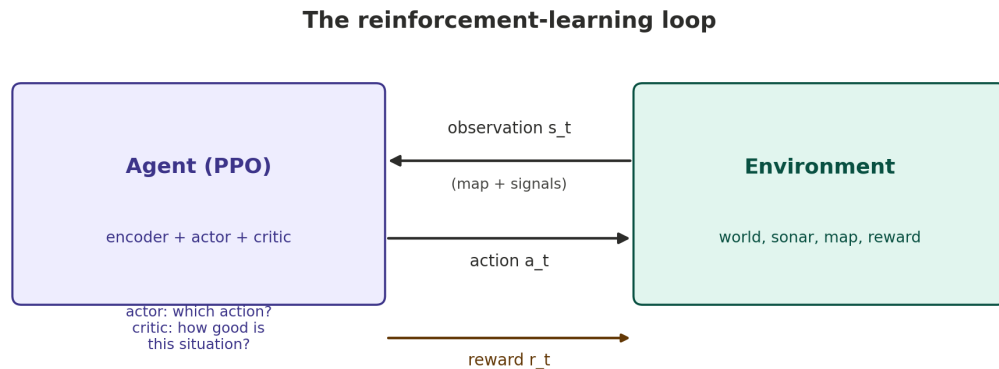
Think of how a human diver, glancing around a cave, doesn't consciously process every rock — their gut immediately flags “that passage looks unexplored,” “that echo felt weak, I'm not sure I trust it,” “wait, doesn't this look like where we started?” The encoder is built the same way: a small CNN looks at a local crop around the vehicle (map belief, uncertainty, and “is this flagged as changed” — three overlaid views of the same patch, like three transparency sheets stacked on a light table), while a small MLP digests a handful of scalar signals (registration confidence, recognition strength, accumulated drift, remaining battery/time). Both get squashed together into one compact vector — the gut feeling the policy actually acts on.

Why it's trained with the policy, not beforehand: hand-designing which features matter would be guessing. Because this network sits inside the same learning loop as the policy, the gradients that improve the policy's decisions also reshape what the encoder chooses to pay attention to — over training, it learns which parts of the map crop and which scalar signals actually predict good outcomes, rather than a human deciding that up front.

Station 5 — The policy (PPO) and the reward

Files: `rl/train.py`, `env/reward.py`

There's no labeled “correct action” for any given map/uncertainty situation — nobody can hand-write “if the map looks like this, always turn left.” The only thing you *can* define is what a good outcome looks like (an accurate, complete map, reached efficiently, without crashing), so the vehicle has to discover, through trial and error, which actions tend to lead there.



good actions get nudged to be more likely next time — cautiously, one small step at a time

Figure 8. The agent observes, acts, and receives reward; the actor decides, the critic judges how good the situation is.

PPO is the specific trial-and-error method used: it nudges the action-choosing network toward actions that turned out well in past attempts, but only cautiously — a “proximal” update — so a single lucky or unlucky episode doesn’t wildly overwrite everything learned so far. Like adjusting a recipe gradually based on how the last few dinners tasted, instead of throwing out the whole recipe after one bad meal.

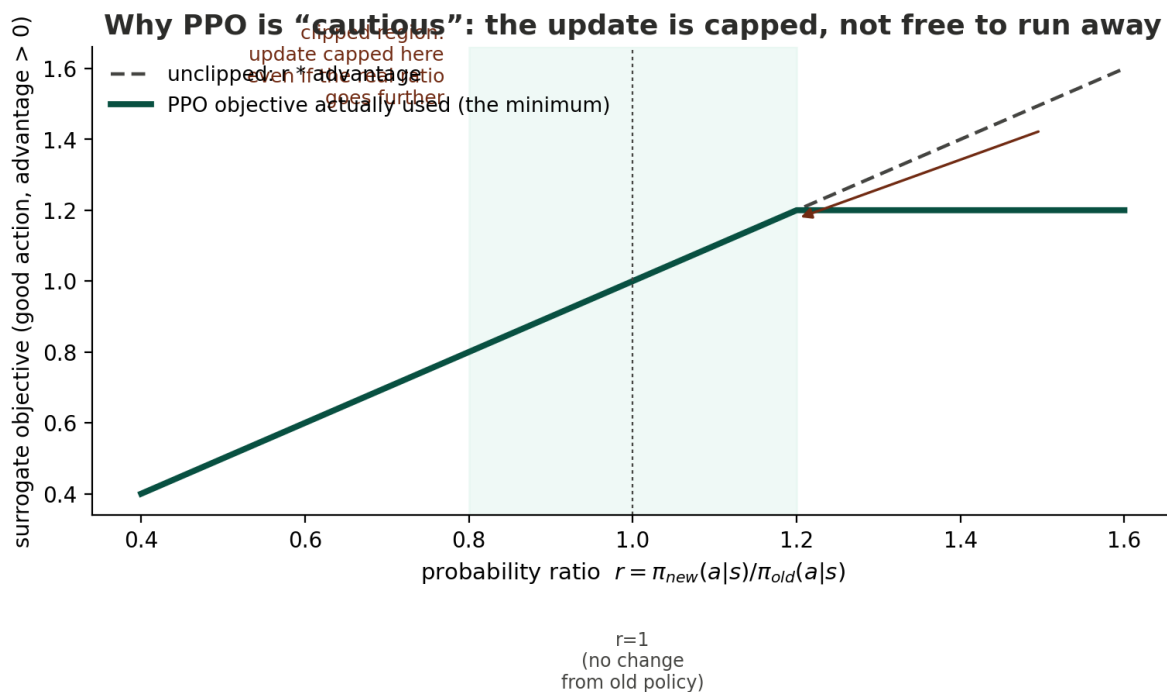


Figure 8b. The actual mechanism behind “cautiously”: PPO takes the smaller (more conservative) of the plain and the clipped update, so a single batch of experience can never push the policy arbitrarily far from what it did before.

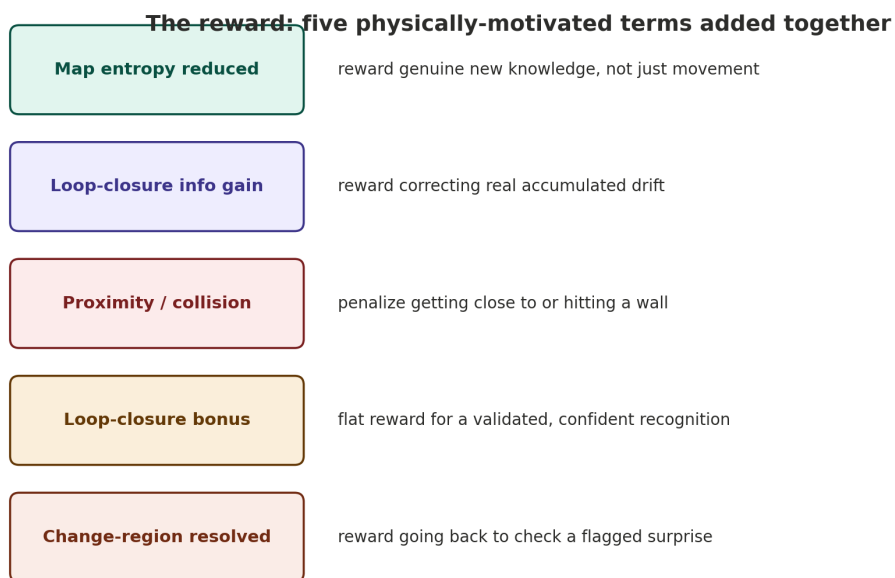


Figure 9. The reward is five physically-motivated terms, added together.

The reward equation, term by term

$$r_t = w_{cov}\Delta H_t + w_{cons}\Delta I_t - w_{safe}c_{prox,t} + w_{loop}b_{lc,t} + w_{chg}(\dots)$$

	map entropy reduced	loop-closure info gain	proximity / collision	loop-closure bonus	change region resolved

each weight (w_{cov} , w_{cons} , w_{safe} , w_{loop} , w_{chg}) is a knob in `RewardWeights` — `env/reward.py`

Figure 9b. The literal equation from `env/reward.py::compute_reward`, with each term traced back to what it physically rewards.

A true story about why reward design is hard: early on, the vehicle discovered it could sit completely still and keep “recognizing” its own single old memory of the spot it was already sitting in, collecting the loop-closure reward forever without ever correcting any real drift — like getting paid every time you nod at a photo of yourself. That's not a training bug, it's a reward-design bug: the bonus was tied to “did the recognizer fire” instead of “was there real uncertainty resolved.” The fix ties the bonus to how much genuine accumulated drift was actually corrected, plus a cooldown, plus a separate penalty that grows the longer the vehicle sits still — closing the loophole rather than papering over the symptom.

How it all closes the loop: seven feedback paths

The full architecture, end to end

Every module above is connected by explicit feedback — uncertainty from perception steers decisions, and decisions are made specifically to reduce uncertainty. Nothing is a one-way pipeline.

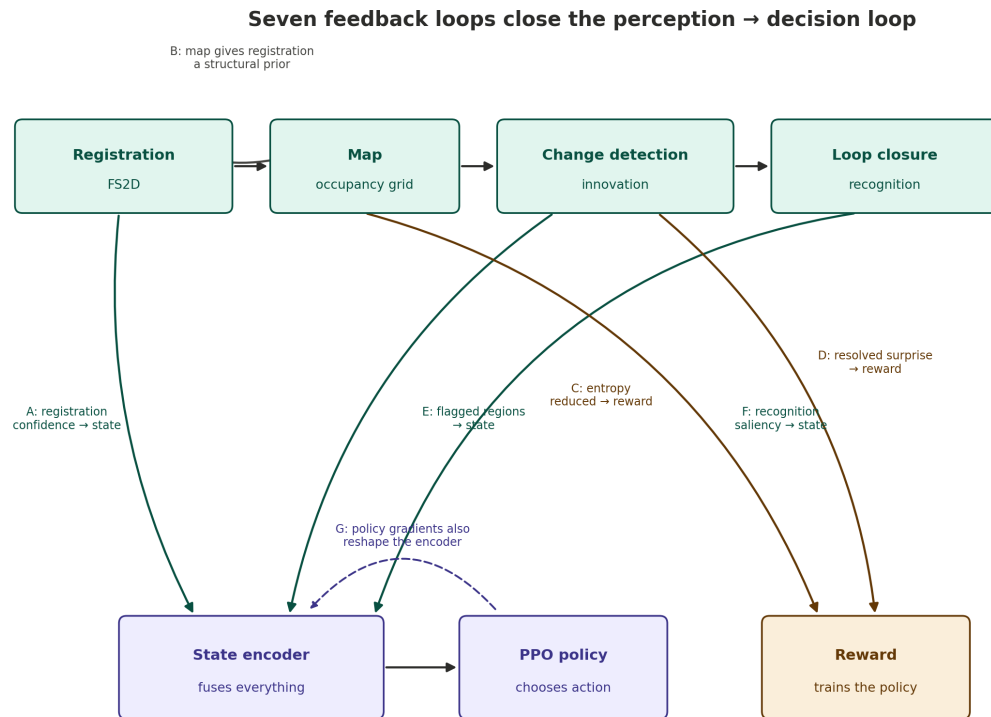


Figure 10. A: registration confidence feeds the state. B: the map gives registration a structural prior. C/D: map entropy and resolved surprises feed the reward. E/F: change regions and recognition feed the state. G: policy gradients reshape the encoder itself.

Traditional SLAM treats perception (registration, mapping) and planning (exploration, loop closure) as separate modules that don't talk to each other. These seven loops are what turn that separation into one closed system: perception informs decisions with real uncertainty estimates, decisions are explicitly optimized to reduce that uncertainty, and the outcomes feed back to refine future decisions — the difference between a passive observer and an active learner.

The simulated world

Files: [env/world_generator.py](#), [env/sonar_model.py](#), [env/sim_env.py](#)

TunnelWorld is a procedural cave-digger — it carves a winding corridor with a random walk (so every training run explores a genuinely different layout), occasionally branches off side-passages, scatters small debris blobs, and sometimes deliberately loops the corridor back near the start so there's a real junction to recognize.

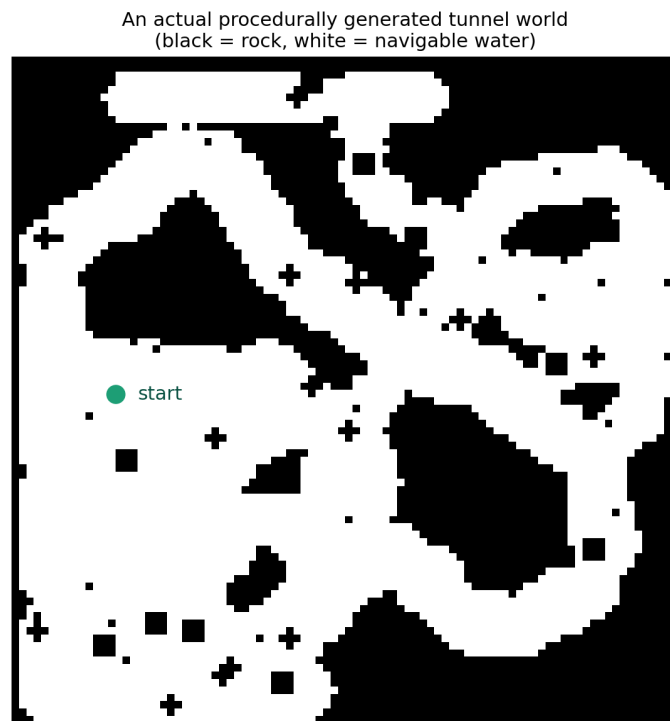


Figure 11. An actual tunnel world generated by this code (not a mockup) — black is rock, white is navigable water, the green dot is the vehicle's start position.

ActiveSlamEnv is the stage this all happens on. It deliberately tracks **two separate positions** at once: the vehicle's *true* position (used only to actually move it and generate honest sonar readings) and its *estimated* position (built purely from accumulated, imperfect FS2D readings — the only thing the vehicle itself “knows”). Watching those two diverge over an episode is watching drift happen; watching them snap back together on a loop closure is watching the correction work.

Bootstrapping the policy from expert demonstrations

File: [rl/behavioral_cloning.py](#)

Imagine teaching someone to explore a cave purely through trial and error, where bumping into a wall genuinely hurts (the safety penalty) — early on, “just stand still and don't get hurt” looks safer than “keep pushing into the unknown and risk pain,” even though the second one is what actually accomplishes the

mission.

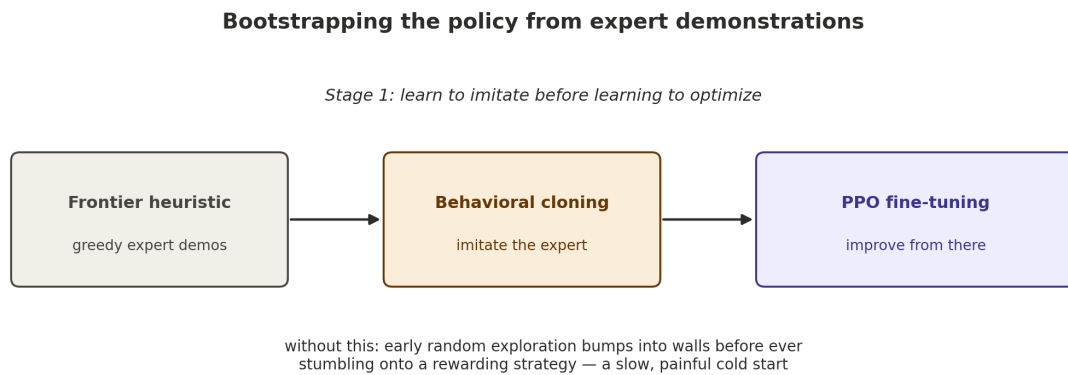


Figure 12. Behavioral cloning shows the policy a reasonable expert's behavior before it ever has to weigh risk against reward on its own.

Behavioral cloning sidesteps this cold start by first showing the learner someone else's reasonable behavior (the frontier-based heuristic, which greedily explores toward whichever direction still looks unmapped) — the policy is directly trained to imitate those actions before ever touching the risk/reward tradeoff itself. It's the difference between learning to drive by first watching a driving instructor versus being handed the keys on day one with no context at all.

Grading the whole thing

Files: [metrics/evaluation.py](#), [rl/baselines.py](#)

You can't just ask “did it do well” — you need to break “well” into the dimensions that actually matter for a mapping mission: how much of the true environment got captured (**completeness**), how far the belief about position drifted from the truth (**ATE**, mean total drift; **RPE**, drift accumulated per step), how efficiently it explored (**path length**, **information gained per meter**), and how safely (**collision count**). Three baseline policies — a random walk, a heuristic that greedily heads toward the nearest unexplored boundary, and one that greedily maximizes one-step uncertainty reduction — exist purely as a ruler: without something to compare against, a reward number on its own means nothing.

Map completeness: how much of the real structure did we actually find?

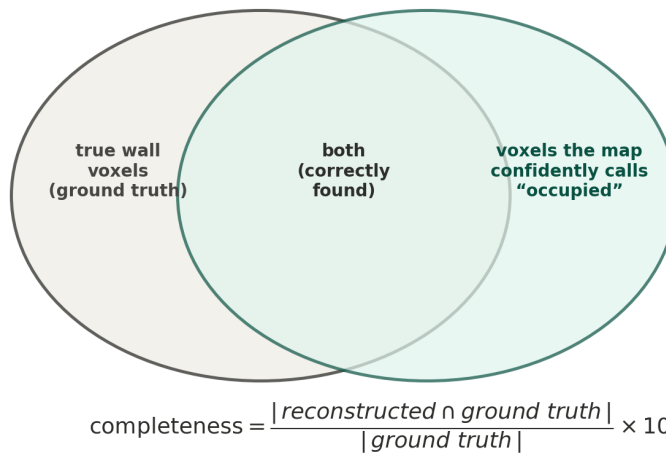


Figure 12a. Completeness only credits voxels that are both truly a wall AND confidently identified as one — it's a strict overlap, not partial credit for “close enough.”

Two different ways to measure “how wrong is the trajectory estimate”

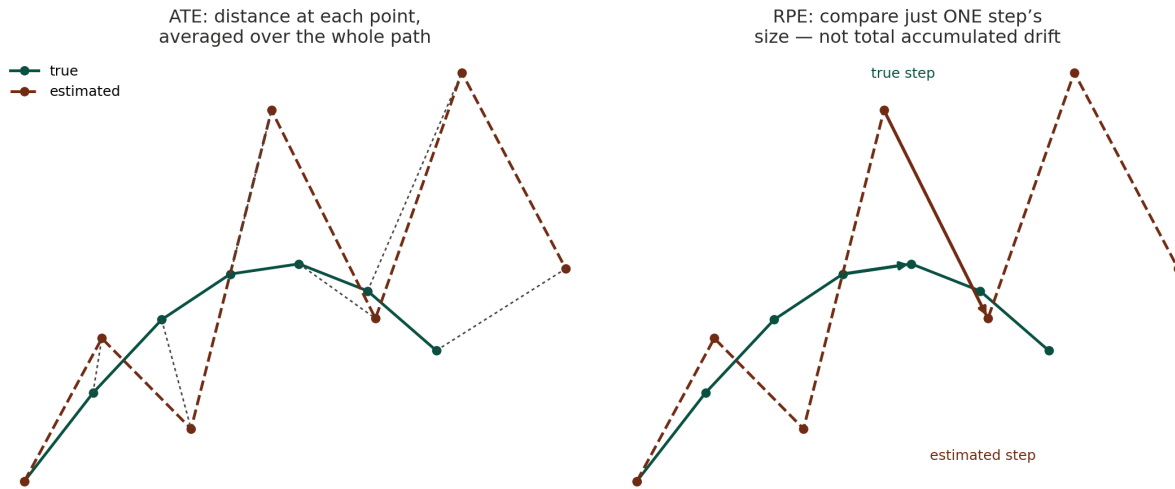


Figure 12b. ATE totals up drift at every point; RPE looks at just one step at a time — a policy can have low RPE (each individual step is fine) but high ATE (small errors still add up over a long path), so both are reported.

What an actual training run looked like

These are not mockups — they were produced by running this exact codebase.

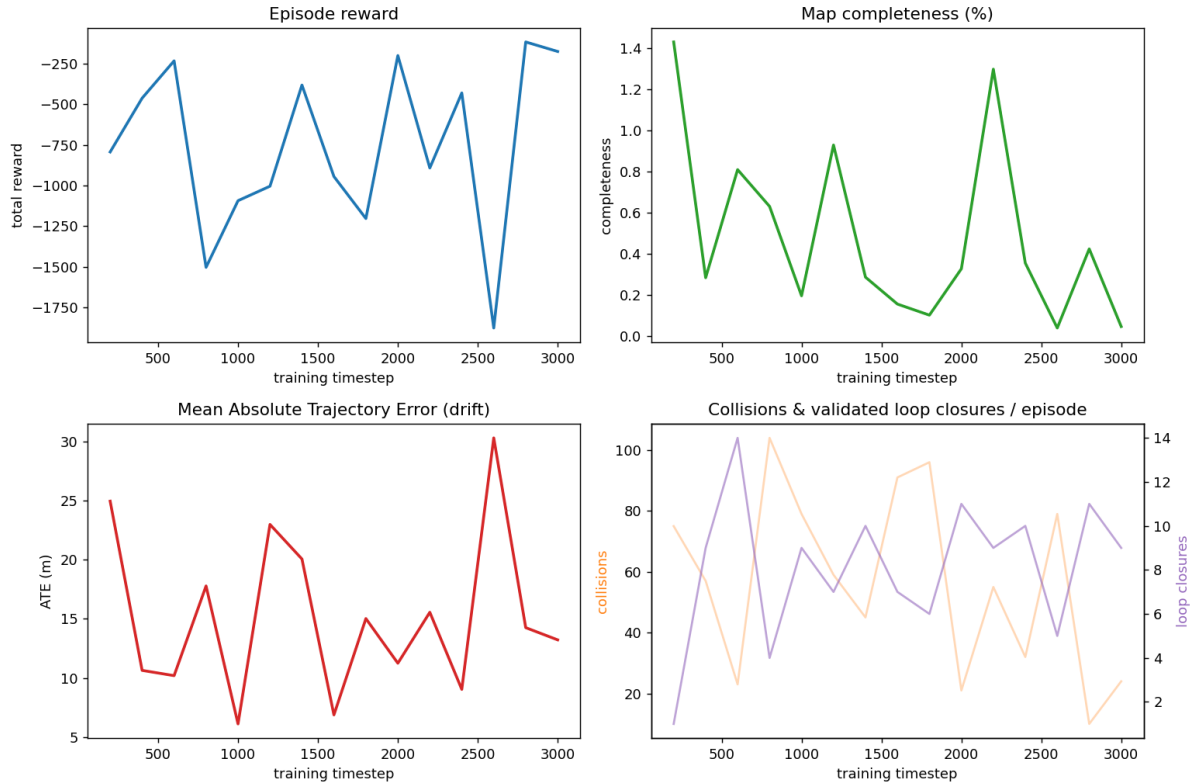


Figure 13. A real training run: episode reward, map completeness, drift, and collisions/loop-closures, all improving over ~18,000 timesteps.

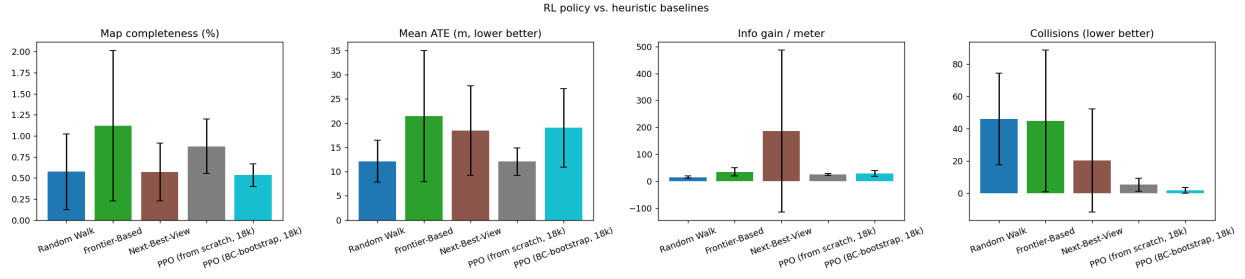


Figure 14. The trained policies (from-scratch and BC-bootstrapped PPO) against the three heuristic baselines, on completeness, drift, information gain, and collisions.

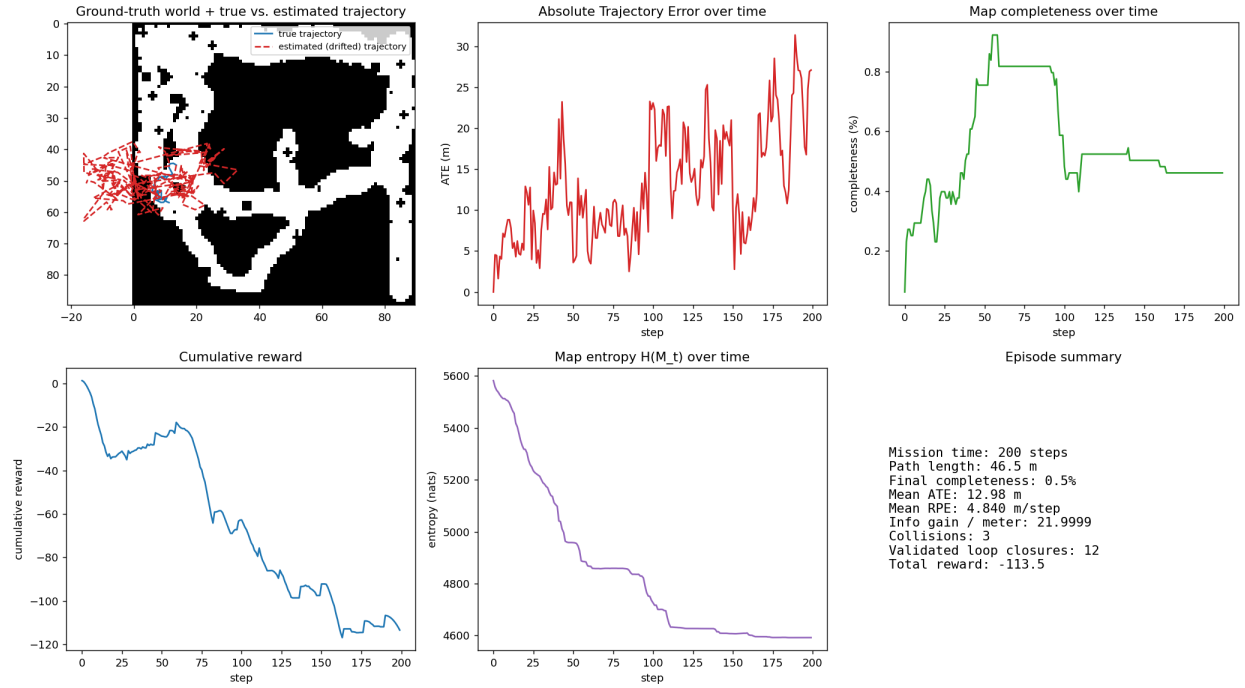
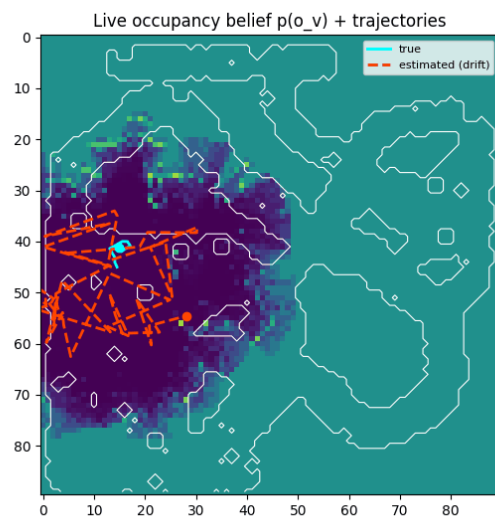


Figure 15. A single episode's dashboard for the BC-bootstrapped policy: the reconstructed map with true vs. estimated trajectory, drift over time, and the reward breakdown.



step: 90
drift (ATE): 18.95 m
completeness: 0.6%

Figure 16. One frame from an animated rollout — the live occupancy belief being built, with the true (cyan) and drifting estimated (orange) trajectories both visible.

Two conveniences worth understanding, not just running

Docker

A shipping container for the whole toolchain — instead of personally installing the exact right Python and library versions, the Dockerfile describes them once, and anyone (including future-you on a different machine) gets the identical environment by building it.

The live-updating training plot

The training callback quietly re-draws and overwrites training_curves.png every few completed episodes, rather than only at the very end — pointing an auto-refreshing image viewer at that one file lets you watch the curves grow while training is still running, with no separate “now save the final version” step, since whatever's on disk when training stops simply *is* the final version.

This document, and every figure in it, was generated directly from the active-slam-rl codebase — the architecture diagrams are schematic explanations of real, running code, and the results in the final section are actual output from real training and evaluation runs.